[Dat Nguyen] C Programing Review

- **Section:** Linux System Programming
- **Topic:** C Programming Language Review
- **Duration:** 3 working days (8 hours/day)

1. Parameter Passing Techniques

1.1 Pass by Value

Passes a copy of the variable to the function. Changes inside the function do not affect the original variable. Memory Note: Allocates - additional memory to store the copied variables.

```
void change(int a) {
    a = 20;
    printf("a inside function: %d\n", a);
}

int main() {
    int a = 10;
    printf("a initialized: %d\n", a);
    change(a);
    printf("a outside function: %d\n", a);
    return 0;
}
```

Output:

```
a initialized: 10
a inside function: 20
a outside function: 10
```

1.2 Pass by Reference

Passes the memory address (pointer) of the variable to the function. Changes made inside the function directly update the original variable. Saves memory because no new variable copies are created.

```
void change(int *a) {
    *a = 20;
    printf("a inside function: %d\n", *a);
}

int main() {
    int a = 10;
    printf("a initialized: %d\n", a);
    change(&a);
    printf("a outside function: %d\n", a);
    return 0;
}
```

Output:

```
a initialized: 10
a inside function: 20
a outside function: 20
```

2. Pointers

A **pointer** is a variable that stores the memory address of another variable.

When pointer `p` points to variable `n`, you can read or modify the value of `n` directly through `p`.

- The value stored at the memory address can change, but the memory address itself remains constant.

```
int main() {
    int n = 10;
    int *p = &n;

    printf("Before changing value\n");
```

```
    printf("Value of n = %d\n", n);
    printf("Address of n: %p\n", (void *)p);

    printf("\nAfter changing value\n");
    *p = 20;
    printf("Value of n = %d\n", *p);
    printf("Address of n: %p\n", (void *)p);

    return 0;
}
```

Output:

```
Before changing value
Value of n = 10
Address of n: 0x7ffeefbff5ac

After changing value
Value of n = 20
Address of n: 0x7ffeefbff5ac
```

2.1 Pointer to Pointer (Double Pointer)

A double pointer stores the address of another pointer.

To store the address of a pointer (Single pointer), we need to declare a pointer to a pointer (Double pointer). Through the second-level pointer, we can also modify the value that the first-level pointer is pointing to.

```
int main(void)
{
    int n = 10;
    int *p = &n;   // Single pointer
    int **pp = &p; // Double pointer

    printf("Value of n: %d\n", n);
```

```
    **pp = 30; // Modify 'n' through double pointer 'pp'
    printf("New value of n: %d\n", n);

    return 0;
```

- p holds the address of variable n.
- pp holds the address of pointer p.

We can modify the value of n using the double pointer pp.

Output:

```
Value of n: 10
New value of n: 30
```

Another example

```
#include <stdio.h>

void change(int **pp, int *new_address)
{
    *pp = new_address;
}

int main()
{
    int a = 10;
    int b = 20;

    int *p = &a;

    printf("Before change:\n");
    printf("a = %d\n", a);
    printf("b = %d\n", b);
    printf("*p = %d\n", *p);

    change(&p, &b);

    printf("\nAfter change:\n");
    printf("a = %d\n", a);
    printf("b = %d\n", b);
```

```
    printf("*p = %d\n", *p);

    return 0;
}
```

Output:

```
Before change:
a = 10
b = 20
*p = 10

After change:
a = 10
b = 20
*p = 20
```

2.2 Void Pointer

A `void` pointer has no associated data type or memory size. Therefore, it can hold the memory address of any variable type, provided it is type-cast when dereferenced.

```
int main()
{
    int a = 19;
    float b = 3.28;
    void *p;

    p = &a;
    printf("%d\n", *(int *)p);

    p = &b;
    printf("%f\n", *(float *)p);

    return 0;
}
```

Output:

```
19
3.280000
```

2.3 Pointer Size

Unlike standard variables, the size of a pointer does not depend on the data type it points to. Instead, it depends on the **system architecture**:

System Architecture	Pointer Size
32-bit system	4 bytes
64-bit system	8 bytes

3 Pointers and Arrays

When a pointer points to an array, it holds the memory address of the array's **first element** (`&array[0]`).

```
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a; // Equivalent to: int *p = &a[0];
    printf("%d\n", *p);       // Output: 10
    printf("%d\n", *(p + 1)); // Output: 20
    printf("%d\n", *(p + 2)); // Output: 30
    return 0;
}
```

Expression	Memory Calculation	Corresponding Array Element
`p`	Base Address	`&arr[0]`
`p + 1`	Base Address $+ 4$ bytes	`&arr[1]`
`p + 2`	Base Address $+ 8$ bytes	`&arr[2]`

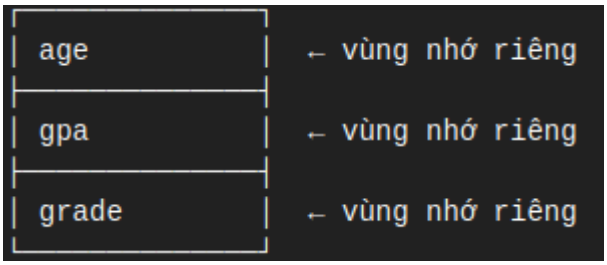
4. Struct, Union

4.1 Struct

A **struct** (structure) is a user-defined data type that allows grouping variables of different data types into a single unit.

- Each member inside a struct is allocated its own memory location.

```
struct Student
{
    int id;
    int age;
    char grade;
};
```



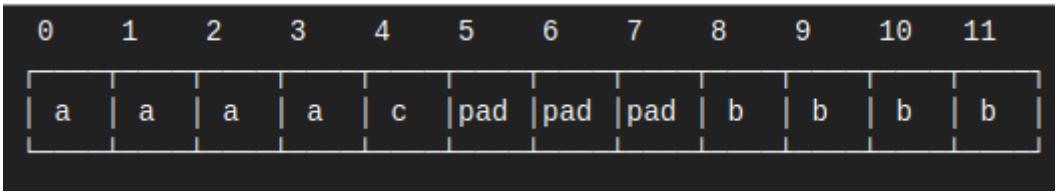
- All members can be accessed simultaneously at any time.
  - The total size of a struct is **at least** the sum of its individual members' sizes.

```
struct Student
{
    int id;    // 4 bytes
    int age;   // 4 bytes
    char grade; // 1 byte
};

int main(void)
{
    printf("Struct size: %lu bytes\n", sizeof(struct Student));
    return 0;
}
```

Output:

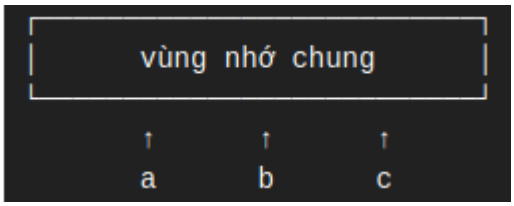
```
Struct size: 12 bytes
```



4.2 Union

A **Union** is similar to a Struct, but with one major difference: **all members share the same memory location**.

```
typedef union {
    int a;
    int b;
    char c;
} Data;
```



- The total size of a Union equals the size of its **largest member**.
- Modifying one member overwrites the values of the other members because they share memory.
- Only **one member** can hold a valid value at any given time.

```
typedef union {
    int a;
    int b;
} Data;

int main()
{
    Data x;
    x.a = 20;
    printf("a = %d\n", x.a);
    printf("b = %d\n", x.b);
    return 0;
}
```

Output:

```
a = 20
b = 20
```

5. Dynamic Memory

Dynamic memory allocation allocates memory at runtime on the **Heap**.

- The `malloc()` function takes the number of bytes to allocate and returns a `void*` pointer pointing to the allocated memory block.
- You must release memory using `free()` after use to prevent memory leaks.

```
malloc()
   |
   ▼
Request memory block
   |
   ▼
Check for NULL
   |
   ▼
Memory allocated ⟶ Use memory
   |
   ▼
Finish using memory
   |
   ▼
free()
   |
   ▼
Release memory back to system
```

```
int main(void)
{
    // 1. Call malloc() to request memory
    int *ptr = (int *)malloc(5 * sizeof(int));
    // 2. Check for NULL
    if (ptr == NULL)
```

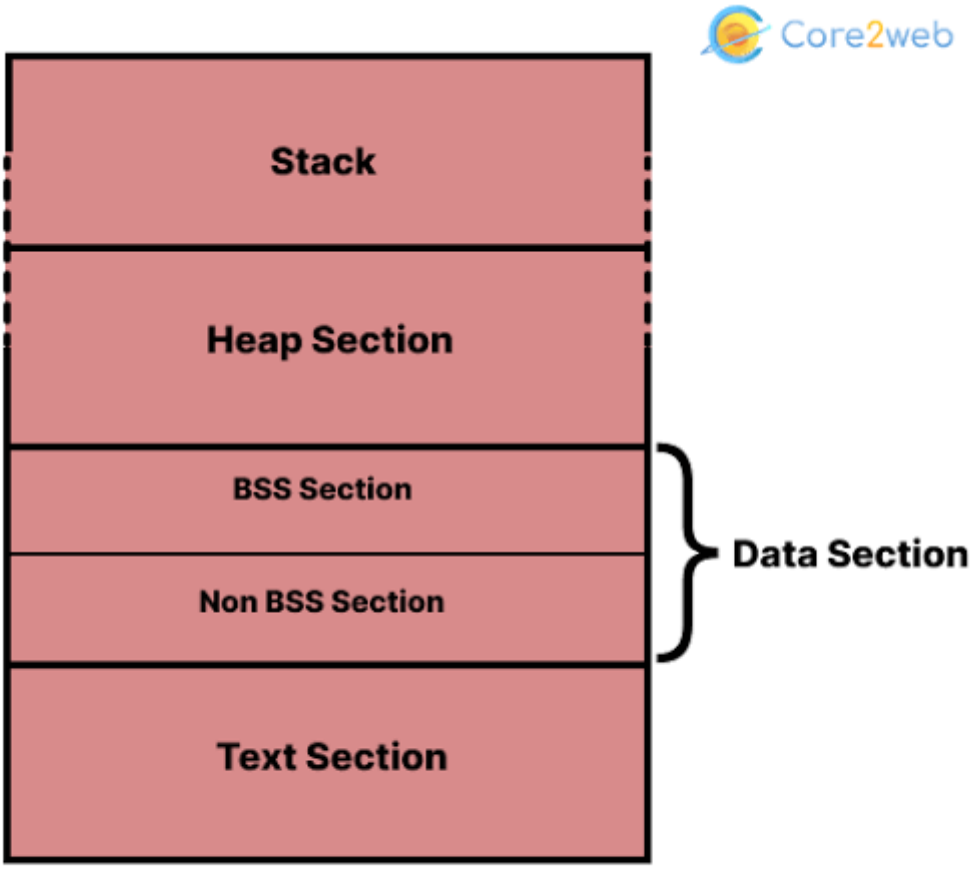
```
    {
        printf("Memory allocation failed!\n");
        return 1;
    }
    // 3. Use memory
    for (int i = 0; i < 5; i++)
    {
        ptr[i] = i * 10;
        printf("ptr[%d] = %d\n", i, ptr[i]);
    }
    // 4. Finish & free()
    free(ptr);
    ptr = NULL; // 5. Release memory and prevent dangling pointer
    return 0;
}
```

Function	Description	Example
`malloc`	Allocates memory, **uninitialized**	`malloc(10 * sizeof(int))`
`calloc`	Allocates memory, **sets all bytes to 0**	`calloc(10, sizeof(int))`
`realloc`	**Resizes** the allocated memory block	`realloc(p, 20 * sizeof(int))`
`free`	**Frees** allocated memory	`free(p)`

Common Memory Pitfalls

Error	Cause	Avoid
Memory leak	`malloc` → forget to `free`	`free()` when no longer used
Double free	`free()` twice	`free()` then `p = NULL`
Dangling pointer	Use pointer after `free()`	`free()` → `p = NULL`

6. Layout memory



When a C program runs, its memory is divided into five main segments:

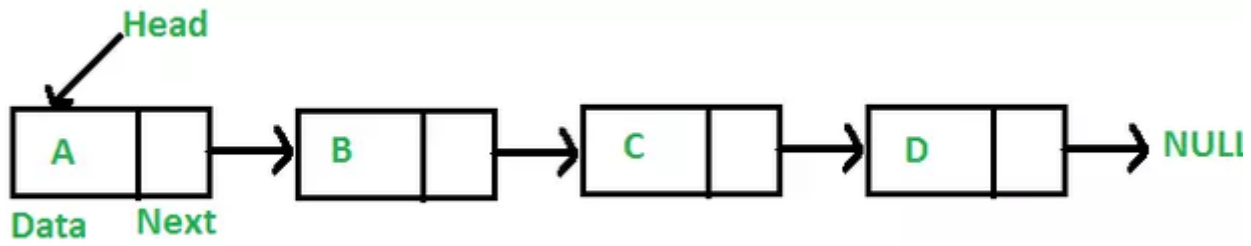
Segment	Purpose
Text Section	Stores compiled machine code (read-only).
Data Section	Stores initialized global and static variables.
BSS Section	Stores uninitialized global and static variables (set to 0).
Heap Section	Allocated at runtime via dynamic memory functions.
Stack Section	Stores local variables, function arguments, and return addresses.

7. Singly/doubly linked lists

7.1 Singly linked lists

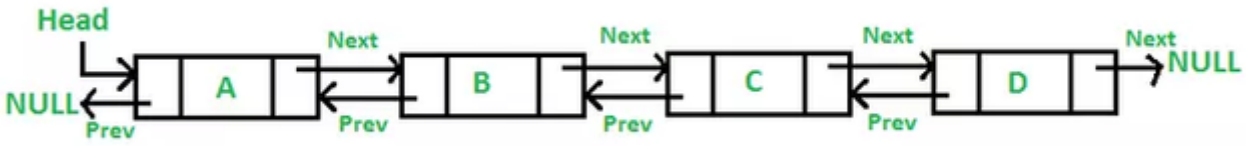
Syntax

```
struct Node {
    int data;          // Stores the data
    struct Node* next; // Pointer to the next node
};
```



- Elements in the linked list are connected by pointers.
- The last element points to NULL.
- The number of elements can dynamically increase or decrease during program execution.
- Memory utilization is optimized.

7.2 Doubly linked list



- Head points to the first node.
- Each node contains data, a Prev pointer, and a Next pointer.
- Next points to the next node.
- Prev points to the previous node.
- The first node's Prev is NULL.
- The last node's Next is NULL.

8. GCC Compilation

A compiler converts source code into an executable file through 4 distinct stages.

Stage	GCC Command	Output File	Description
Preprocessing	`gcc -E main.c -o main.i`	`main.i`	Expands macros, includes header files, and removes comments.
Compilation	`gcc -S main.i -o main.s`	`main.s`	Translates the preprocessed C code into Assembly language code.
Assembly	`gcc -c main.s -o main.o`	`main.o`	Converts Assembly code into machine code (Object file / Binary).
Linking	`gcc main.o -o main`	`main` (or `main.exe`)	Combines object files and libraries into a single executable file.

9. LAB

Link to lab

LAB1

Implement a singly linked list in C from scratch (insert/delete/traverse) to confirm pointer and dynamic-memory fluency

Source code

```
int main(){
    node *head = NULL;
    pushFront(&head, 6);
    pushFront(&head, 5);
    pushFront(&head, 4);
    pushFront(&head, 3);
    pushFront(&head, 2);
    pushFront(&head, 1);
    pushBack(&head, 7);
    printf("Initial list: ");
    duyet(head);
    insert(&head, 100, 6);
    printf("After inserting 100 at position 6: ");
    duyet(head);
```

```
    popFront(&head);
    printf("After removing the first node: ");
    duyet(head);
    popBack(&head);
    printf("After removing the last node: ");
    duyet(head);
    erase(&head, 2);
    printf("After removing node with value 2: ");
    duyet(head);
    return 0;
}
```

Output:

```
Initial list: 1 2 3 4 5 6 7
After inserting 100 at position 6: 1 2 3 4 5 100 6 7
After removing the first node: 2 3 4 5 100 6 7
After removing the last node: 2 3 4 5 100 6
After removing node at position 2: 2 4 5 100 6
```

LAB2

Compile a small program through each gcc stage (-E, -S, -c, then full link) and inspect the output of each stage

Source code

LAB3

Write a small program that uses function pointers to implement a simple callback/dispatch table

Source code

```
int main(){
    int a = 12, b = 4;
    execute(mfunction[ADD], a, b);
    execute(mfunction[SUBTRACT], a, b);
    execute(mfunction[MULTIPLY], a, b);
    execute(mfunction[DIVIDE], a, b);
```

```
    return 0;
}
```

Output:

```
Result = 16
Result = 8
Result = 48
Result = 3
```

Revision #19
Created 2026-08-19 07:19:34 UTC by Dat Nguyen
Updated 2026-08-21 02:58:46 UTC by Dat Nguyen